基于延迟绑定的eBPF虚拟化架构实践

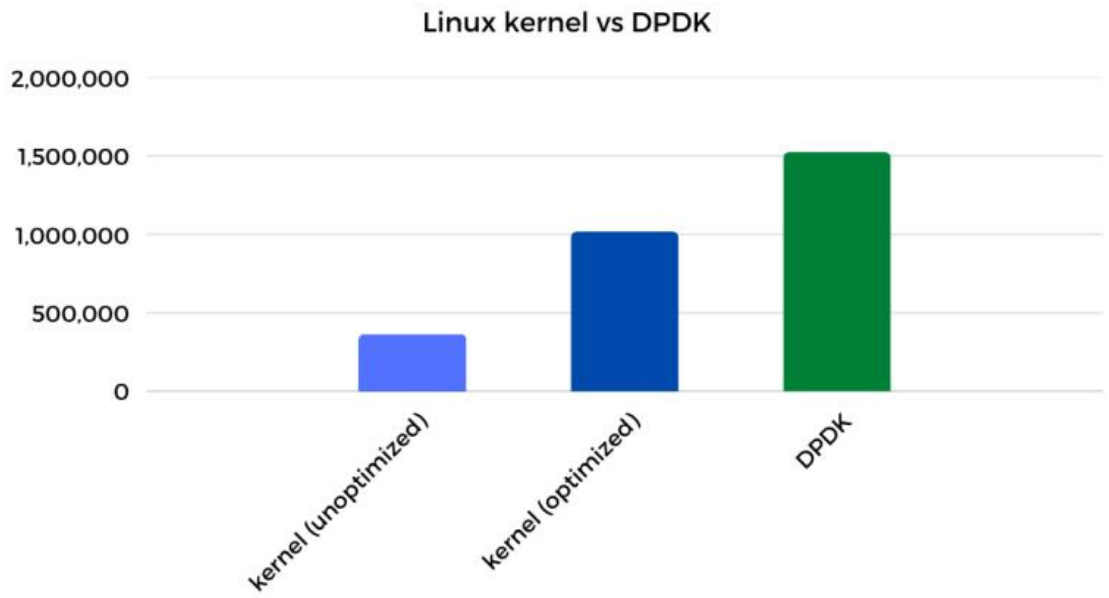
The 4th eBPF Developer Conference (eBPFDC 2026)

张璟
上海交通大学

陕西・西安　　2026年4月18日

千人千面: 定制化的无限可能

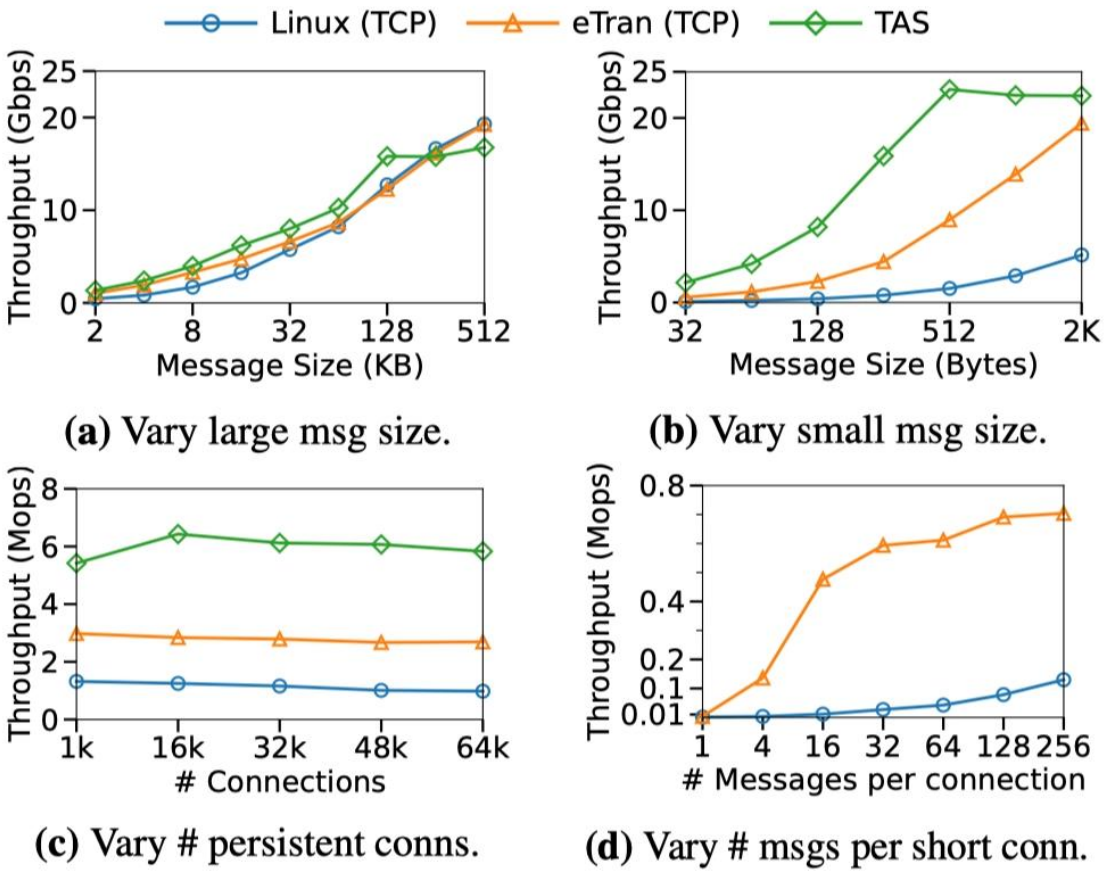
- 专用胜于通用
- DPDK vs. kernel stack



https://talawah.io/blog/linux-kernel-vs-dpdk-http-performance-showdown/

千人千面: 定制化的无限可能

- 专用胜于通用
- AF_XDP 网络栈



(a) Vary large msg size.

(b) Vary small msg size.

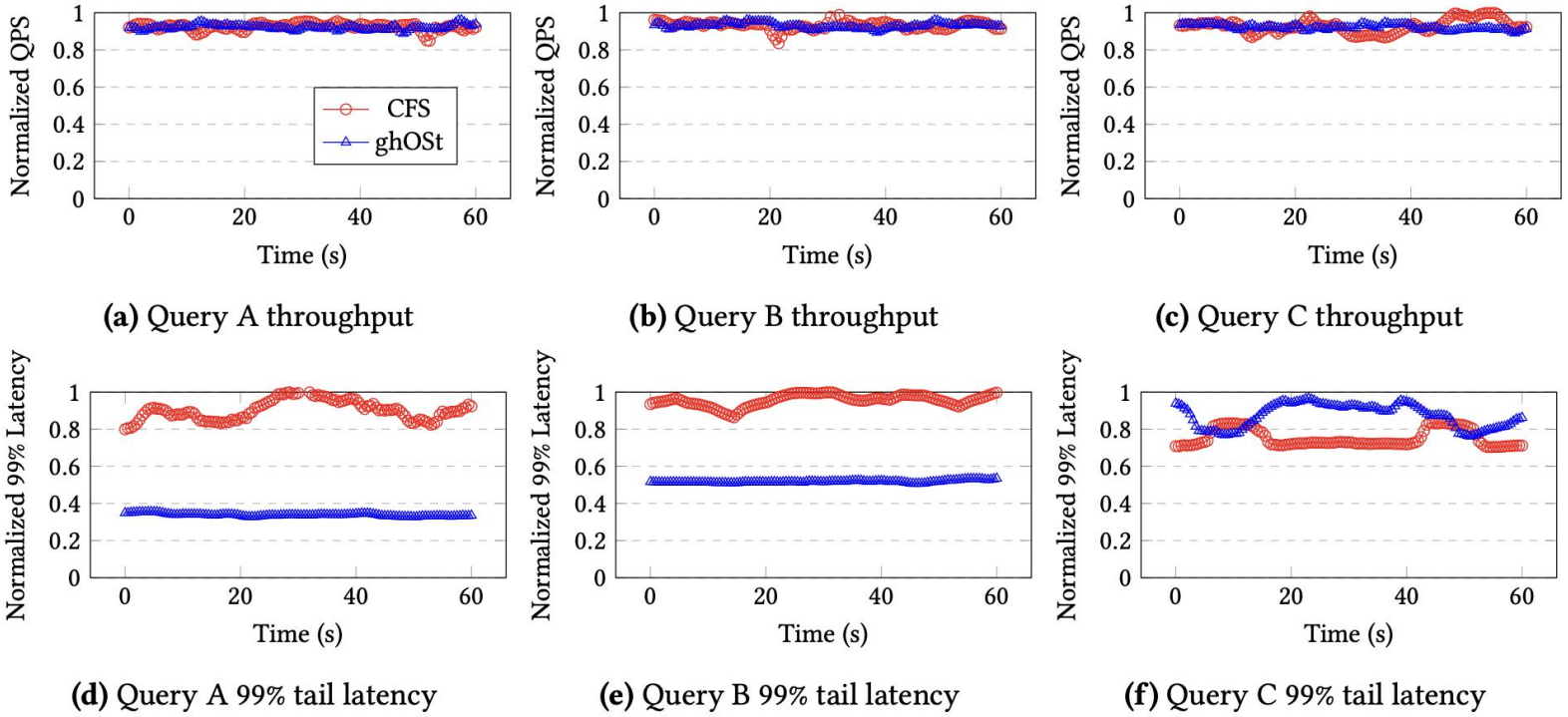
(c) Vary # persistent conns.

(d) Vary # msgs per short conn.

Figure 7: Comparison of message throughput for the TCP echo. Figure (a) has a single outstanding message, while Figure (b) has 64. TAS fails to achieve stable throughput for Figure (d), thus omitted.

千人千面: 定制化的无限可能

- 专用胜于通用
- Scheduler



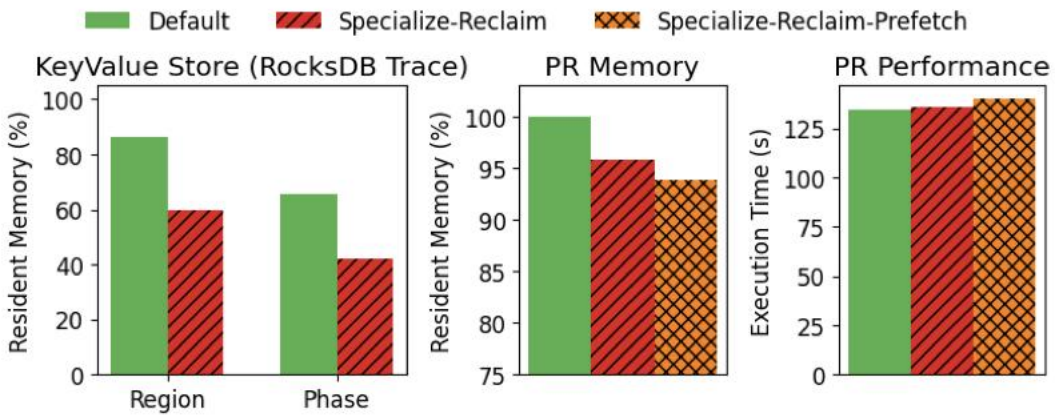
(a) Query A throughput **(b)** Query B throughput **(c)** Query C throughput

(d) Query A 99% tail latency **(e)** Query B 99% tail latency **(f)** Query C 99% tail latency

Figure 8. Google Search benchmark results with CFS and ghOSt scheduling.

千人千面: 定制化的无限可能

- 专用胜于通用
- 内存管理



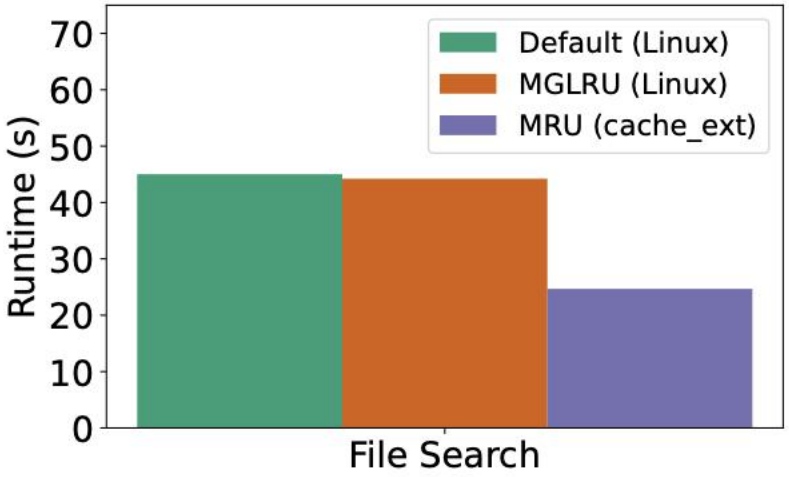
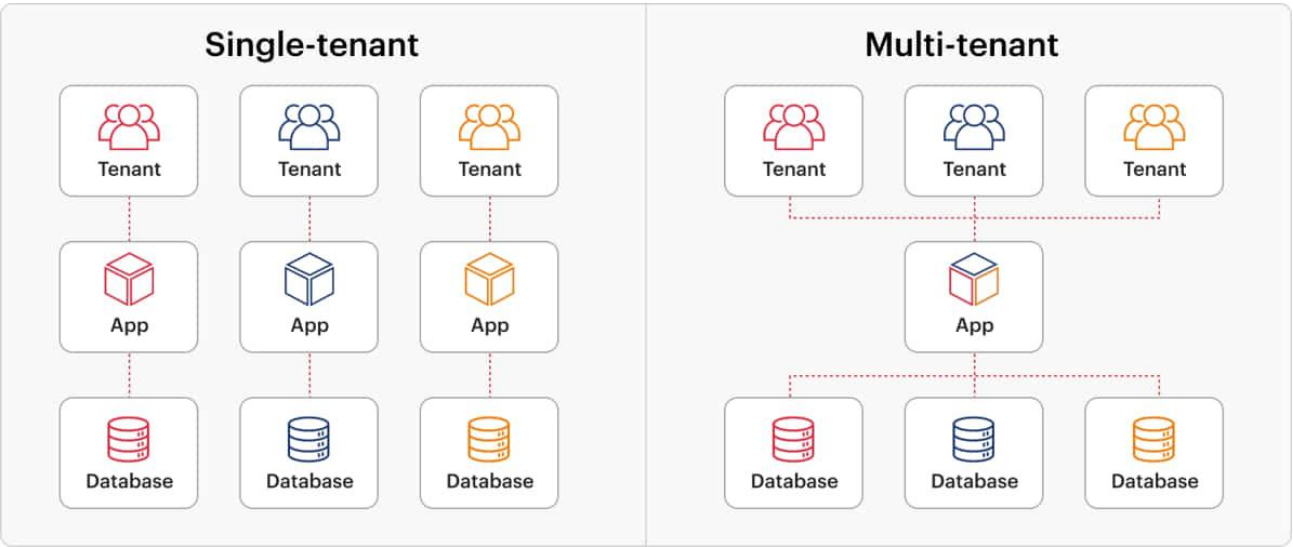


Figure 9. File search workload results (MRU policy).

场景变化：从独占到共享

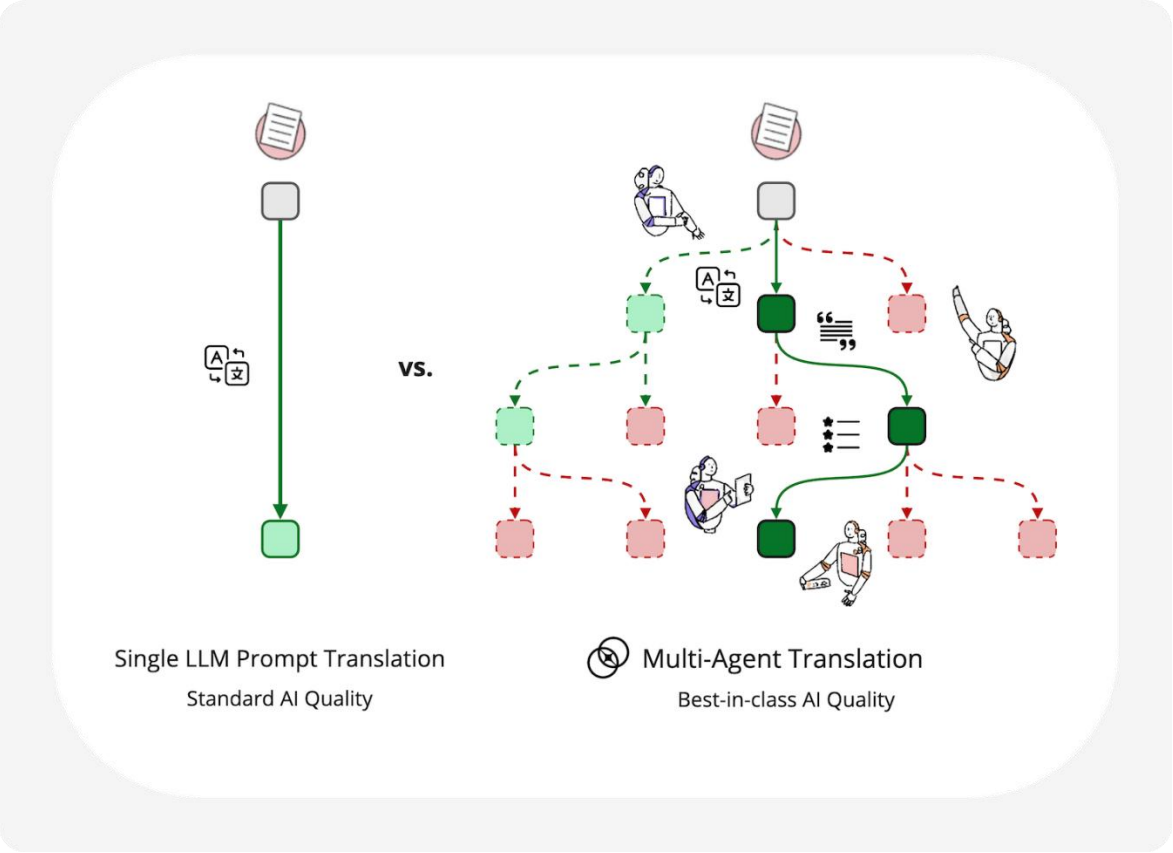
- 云时代多租户
- 提升资源利用率
- 利用异构算力
- 降低部署成本



https://www.sigmoid.com/blogs/powering-businesses-with-a-faster-configurable-multi-tenant-architecture/

场景变化：从独占到共享

- 多 Agent 协作
- 独立上下文
- 独立的执行环境



https://www.onesky.ai/blog/multi-agent-system

为什么 eBPF 还不够?

- Hook 绑定
- 回调触发
- 专用接口交互

Challenge#1: Singleton

- 有些 hook 只能接受一个程序
- struct_ops
- sched_ext
- eBPF iterators

```
SEC(".struct_ops")
struct tcp_congestion_ops
        ebpf_conflict_ops_A = { …};
```

```
SEC(".struct_ops")
struct tcp_congestion_ops
        ebpf_conflict_ops_B = { …};
```

```
libbpf: map 'ebpf_conflict_ops_A': created successfully, fd=3
libbpf: map 'ebpf_conflict_ops_B': created successfully, fd=15
Loading complete. Inserting first struct_ops object...
Attached first using struct_ops map ebpf_conflict_ops_A
Loading complete. Inserting second struct_ops object...
second: attach ebpf_conflict_ops_B failed (17): File exists
```

Challenge#2: Conflict

- 彼此之间产生功能冲突
- 共享变量、返回值

```
SEC("tc/ingress") // container-A redirect to egress
int tc_ingress(struct __sk_buff *ctx) {
    if (iph.saddr == 0x9c3ca8c0) {
        iph.saddr = 0x0A0A0A01;
        return bpf_redirect(2, 0);
    }
    return TC_ACT_PIPE;
}
SEC("tc/egress") // container-B redirect back
int tc_egress(struct __sk_buff *ctx) {
    if (iph.saddr == 0x0A0A0A01)
        return bpf_redirect(2, 1);
    return TC_ACT_PIPE;
}
```

Listing 1: **Infinite forwarding loop caused by functionality conflicts**. *Two tenants in TC redirect the same packets.*

Challenge#3: Interference

- 不同用户之间产生性能干扰
- 0 ⇢ 1 的影响最大
- 不同的 hook 机制之间差异没那么大
- 某些 hook 存在放大效应

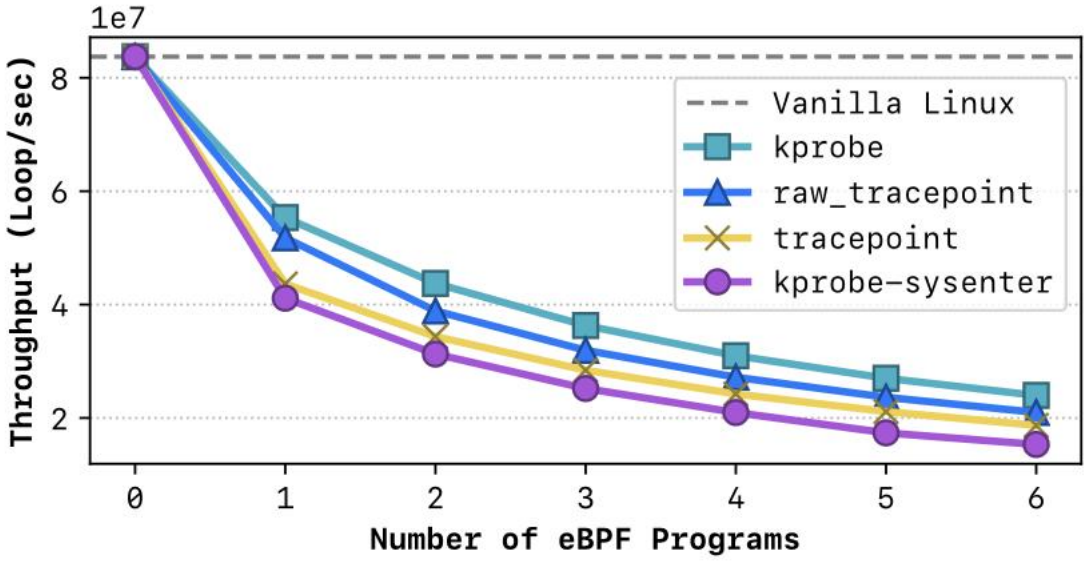


Figure 3: ***Performance interference on UnixBench [90].*** *The Y-axis represents syscall throughput (loops/second). Performance degraded when attaching eBPF programs to the getpid syscall. kprobe-sysenter attaches to* `sys_enter` *and relies on in-program conditional checks to filter for getpid, similar to tracepoint and raw_tracepoint.*

为什么现有的方案不够?

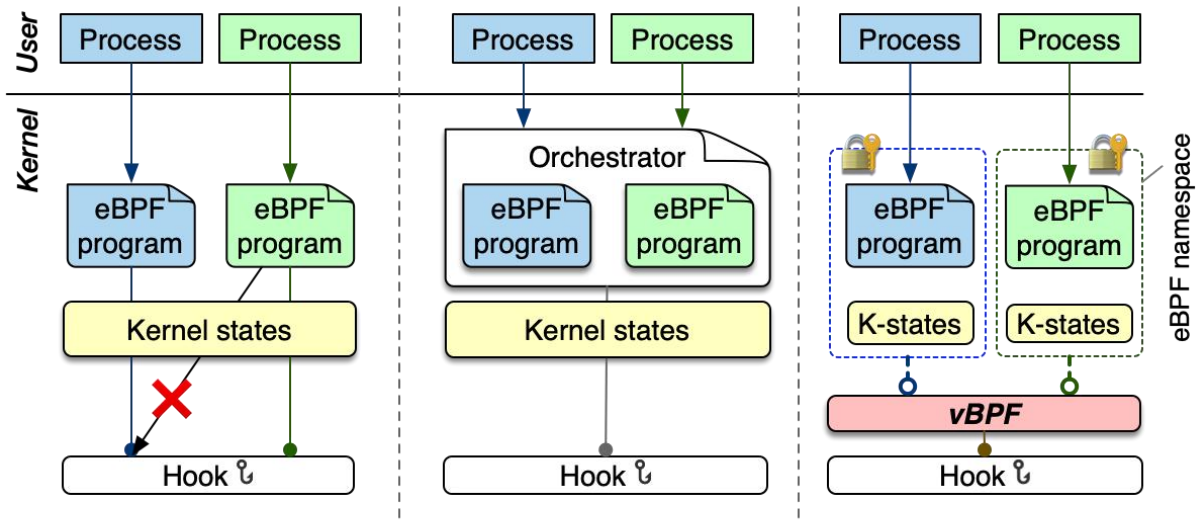
- Conditional Check
- Cgroups-bpf
- Orchestrators

Approaches	Enforced Isolation	State Isolation	Global Scope	Verifier Scalability	Trans-parency
Conditional Checks [49]	✗	✗	✓	✓	✗
Cgroups BPF [84, 105]	✓	✗	✗	✓	✓
Orchestrators [6, 7, 41]	✓	✗	✓	✗	✗
vBPF *(Our work)*	✓	✓	✓	✓	✓

Table 1: ***Existing approaches to eBPF multi-tenanccy.***

静态绑定的语义鸿沟

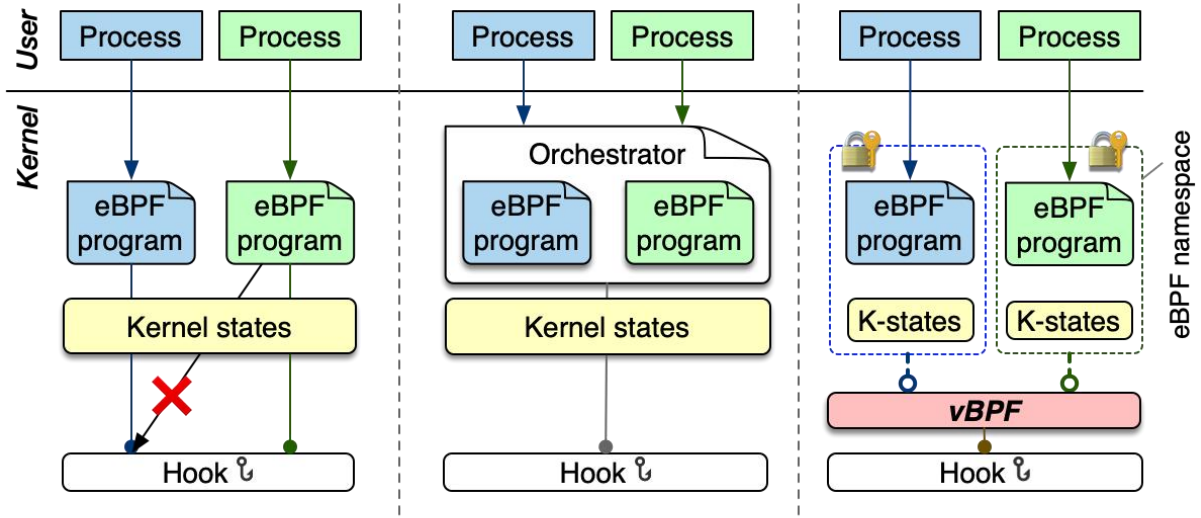
- 程序加载时绑定 hook
- 事件在运行时触发 hook



(a) Native Static Binding (b) Orchestrator Static Binding (c) Dynamic Binding (Ours)

Insight: Late-Binding

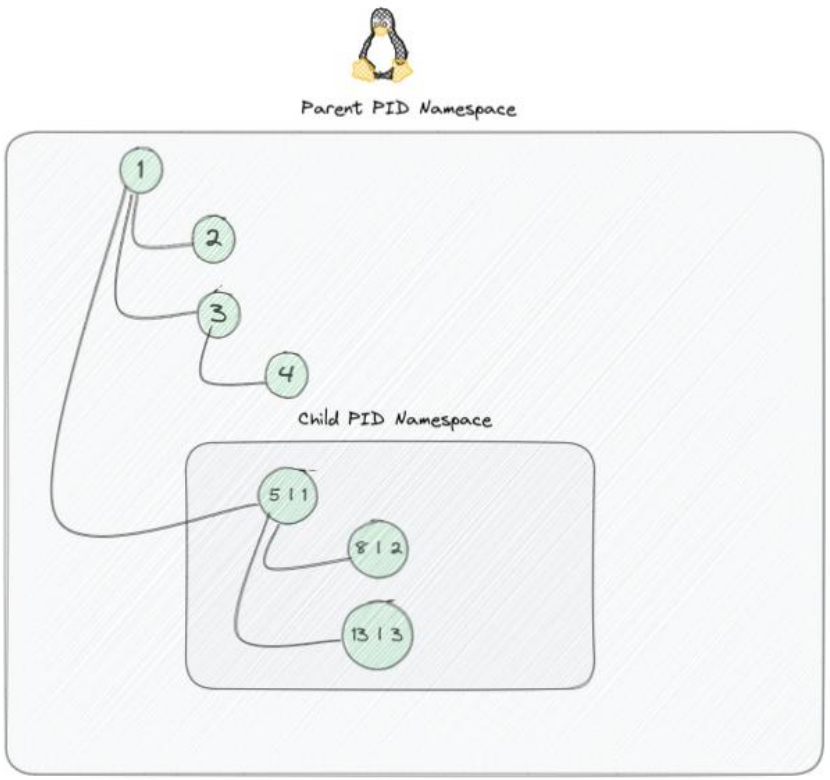
- 既然事件是动态触发的
- Hook和程序的关联应该是动态的
- physical hook 不再直接承载业务逻辑，而是退化为统一拦截入口



(a) Native Static Binding (b) Orchestrator Static Binding (c) Dynamic Binding (Ours)

eBPF namespace

- 如何在eBPF层面表达用户的概念?
- 复用 namespace!
- 复用一切相同的接口和上层生态
- Task 和 eBPF namespace 关联



Design#1: vBPF Sniffer

- 挑战：eBPF namespace缺失
- 进程上下文：用户态进程invoke syscall
  - current task 往往能直接映射到租户
- 中断上下文：一般由硬件触发
  - 此时的current task只代表“被中断”的进程

Design#1: vBPF Sniffer

- 观察：
- 中断事件与先前在进程上下文中设置的资源相关联。
- e.g.，网络连接和网络包头 4-tuple

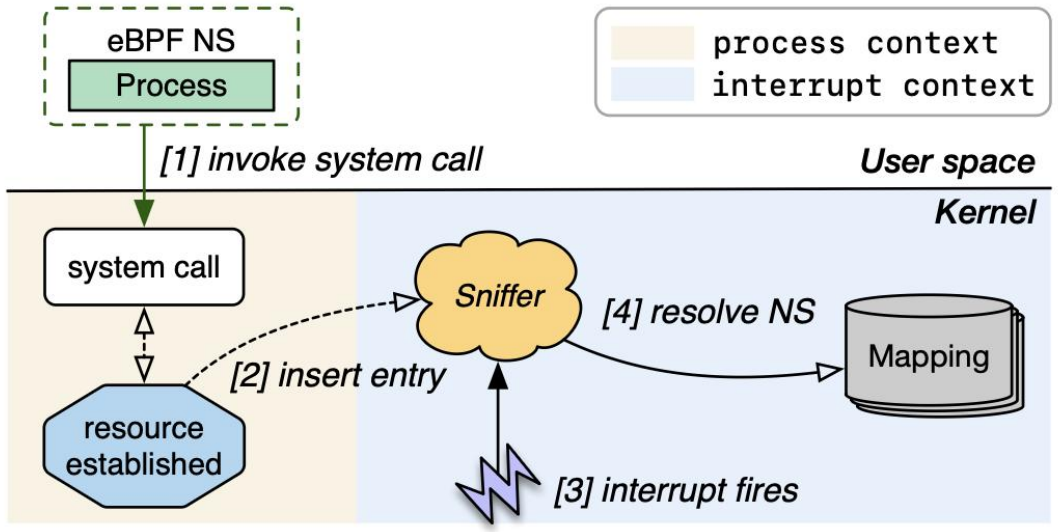


Figure 4: vBPF **Sniffer workflow.** *First, a process invokes a system call ([1]), triggering the Sniffer to record the mapping between the established resource and the namespace ([2]). Later, when a hardware interrupt fires ([3]), the Sniffer uses this pre-recorded mapping to resolve the associated namespace ([4]).*

Design#1: vBPF Sniffer

- 两阶段归属恢复机制

[1] context-aware:

在 bind/connect/请求创建等进程路径上建立 {resource -> ns} 映射

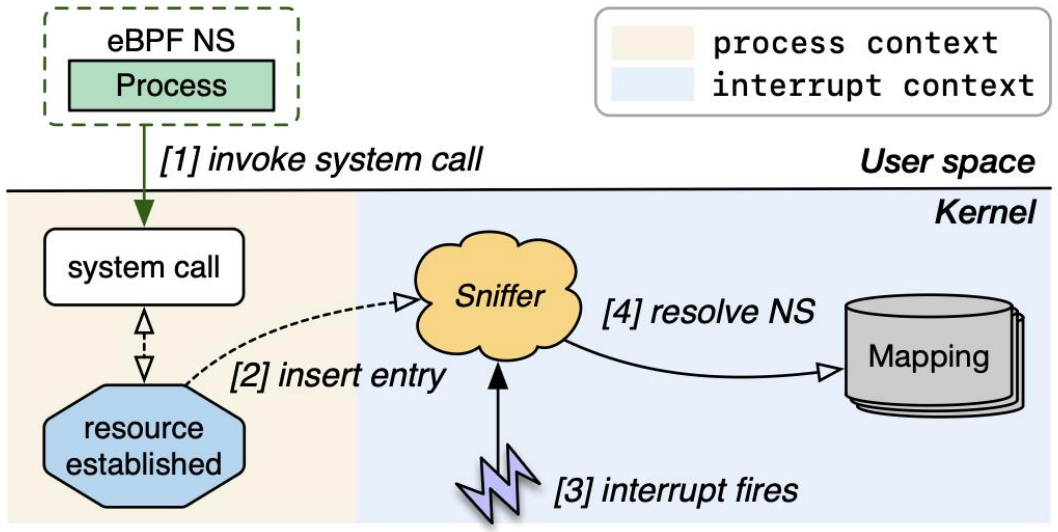


Figure 4: vBPF **Sniffer workflow.** *First, a process invokes a system call ([1]), triggering the Sniffer to record the mapping between the established resource and the namespace ([2]). Later, when a hardware interrupt fires ([3]), the Sniffer uses this pre-recorded mapping to resolve the associated namespace ([4]).*

Design#1: vBPF Sniffer

- 两阶段归属恢复机制

[2] context-free:

中断触发时从事件元数据恢复同一key，再反查出真实租户。

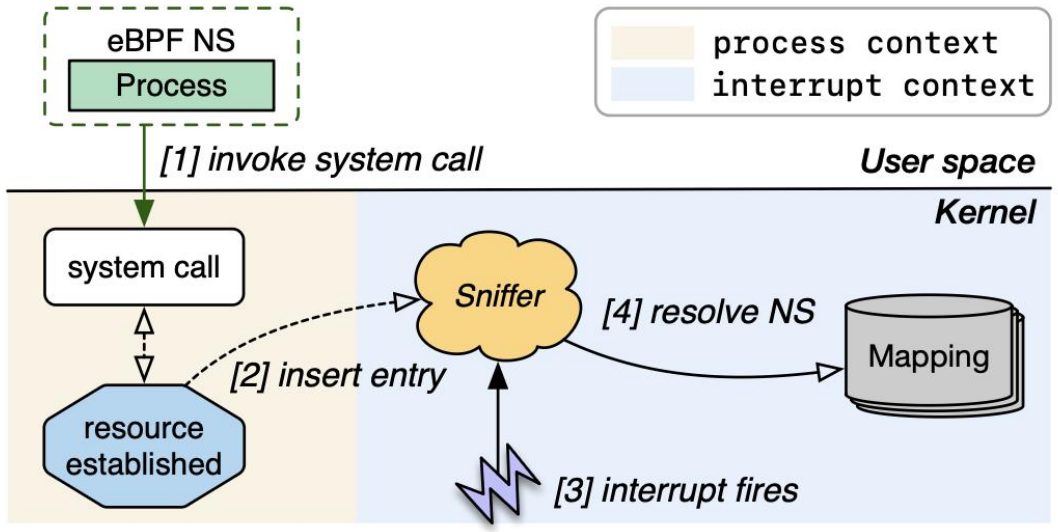


Figure 4: vBPF **Sniffer workflow.** *First, a process invokes a system call ([1]), triggering the Sniffer to record the mapping between the established resource and the namespace ([2]). Later, when a hardware interrupt fires ([3]), the Sniffer uses this pre-recorded mapping to resolve the associated namespace ([4]).*

Design#1: vBPF Sniffer

- 统一接口与抽象边界
- writer/reader 分离使不同对象结构能汇聚到一致语义键

Component	API	Description
Sniffer	`insert(writer)`	Extracts a stable key from *writer* and maps it to the current namespace. Called in process context.
	`resolve(reader)`	Looks up the target namespace using a key derived from *reader*. Called in interrupt context.
	`remove(key)`	Deletes the mapping associated with *key* to release resources. Called in process context.
Dispatcher	`get_namespace()`	Identifies the tenant namespace by inspecting the current process context or querying the Sniffer.
	`dispatch(ns)`	Locates the program set for namespace *ns* and invokes the appropriate tenant eBPF programs.
	`attach/detach(ns, prog)`	Registers or unregisters *prog* for namespace *ns*. Updates reference counts to trigger `enable/disable`.
	`enable/disable(ns)`	Lazily initializes the Dispatcher hook on the first attachment, or tears down on the last detach.

Table 2: Core abstractions of vBPF *Sniffer* and vBPF *Dispatcher*.

Design#1: vBPF Sniffer

- 覆盖对象与作用域控制

- 网络类：socket、xdp_md、sk_buff 使用 4-tuple 信息
- 存储类：bio/request 在提交与完成路径之间保持请求 owner 的可追溯性
- 任务类：task_sniffer 处理 task exit 边界，避免任务回收时失去租户信息
- 平台级中断如 IPI、设备热插拔等无 tenant 语义的事件直接落回 host

Design#2: vBPF Dispatcher

- 为 hook 设计“虚函数”
- 哈希处理不同的分支
- 反向链接重建父子关系
- 扁平化层级关系

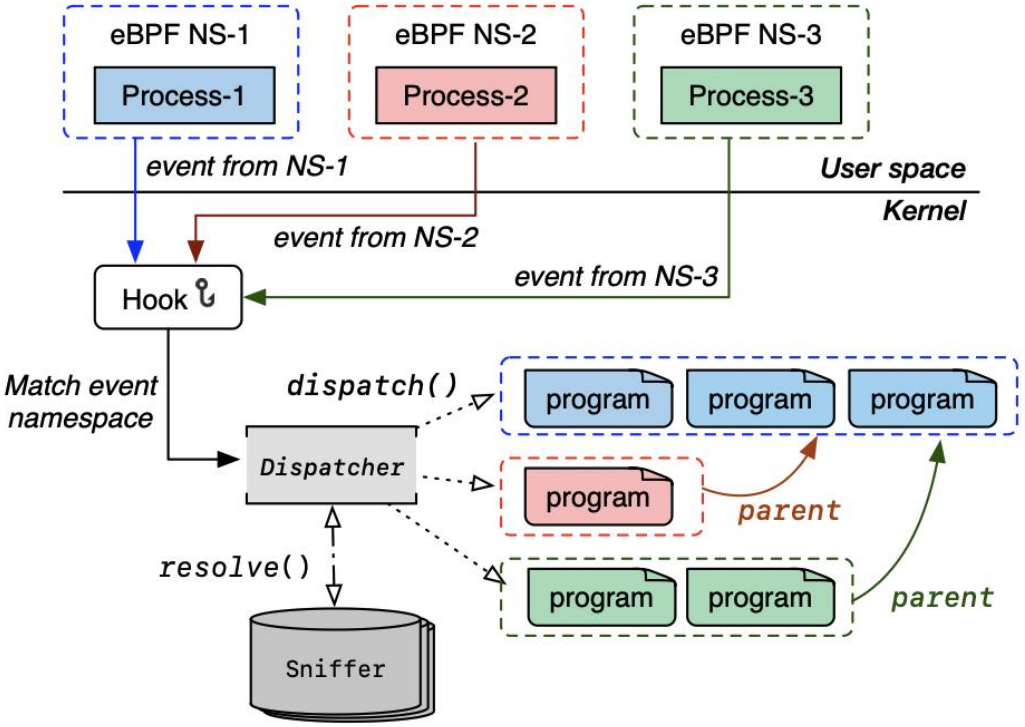


Figure 5: vBPF ***Dispatcher.***

Design#3: Variable Library

- 前面保证了 eBPF 执行的虚拟化，如何保证状态修改的隔离呢?
- 我们提出了一套变量库来保证修改的安全

Design#3: Variable Library

- 观察: eBPF 的设计中，对内核状态的访问本就是受限的（e.g., BPF_CALL_* 宏）
- 解法: 接口有限，因此可以采用编译期静态分析来分析这些接口
- Attribute:
  - *vbpf_helper*: 需要追踪的接口
  - *vbpf_safe*: 开发者手动标注的信任基
- 静态分析器选取这些接口函数，进行内存分析，并结合手动的safe标注

Design#3: Variable Library

- 人工分析是否会占据大部分？如何减轻负担？
- 观察#1: 大部分的 false positive 都来自指针分析
- 观察#2: eBPF 还有一个额外的特性——verifier
- Verifier 为了验证，对不同的helper function的指针打上了不同的标记，我们借助这些标记实现了静态分析的简化

Design#3: Variable Library

- 如果不通过，要如何修改才能保证安全呢?
- 分类：tenant-private state 和 shared state
- 前者本质是 false sharing
- 后者则是真共享
- 因此，我们可以分而治之

Design#3: Variable Library

- eBPF Map 算什么?
- False sharing?

Design#3: Variable Library

- eBPF Map 算什么?
- False sharing?
- 在load-time 确定，但是全局暴露
- 只要能得到 fd 就可以访问

```
+mrzleo@turingine ~ % sudo bpftool map
12: hash_of_maps  name cgroup_hash  flags 0x0
        key 8B  value 4B  max_entries 2048  memlock 165632B
        pids systemd(1)
14: cgroup_array  name cgroup_map  flags 0x0
        key 4B  value 4B  max_entries 1  memlock 312B
        pids systemd-network(551)
15: ringbuf  name written_sysctls  flags 0x0
        key 0B  value 0B  max_entries 262144  memlock 275776B
        pids systemd-network(551)
17: array  name .rodata.str1.1  flags 0x80
        key 4B  value 5B  max_entries 1  memlock 312B
        frozen
        pids systemd-network(551)
34: array  name kprobe_b.bss  flags 0x400
        key 4B  value 4B  max_entries 1  memlock 8192B
        btf_id 258
        pids kprobe(1721748)
35: array  name kprobe_b.rodata  flags 0x80
        key 4B  value 133B  max_entries 1  memlock 440B
        btf_id 258  frozen
        pids kprobe(1721748)
38: array  name libbpf_global  flags 0x0
        key 4B  value 32B  max_entries 1  memlock 336B
        btf_id 258
39: array  name pid_iter.rodata  flags 0x480
        key 4B  value 4B  max_entries 1  memlock 8192B
        btf_id 269  frozen
        pids bpftool(1721865)
40: array  name libbpf_det_bind  flags 0x0
        key 4B  value 32B  max_entries 1  memlock 336B
        btf_id 269
```

Design#3: Variable Library

- eBPF Map 算什么?
- 资源视角?
- 是不是很像 pid?

```
+mrzleo@turingine ~ % sudo bpftool map
12: hash_of_maps  name cgroup_hash  flags 0x0
        key 8B  value 4B  max_entries 2048  memlock 165632B
        pids systemd(1)
14: cgroup_array  name cgroup_map  flags 0x0
        key 4B  value 4B  max_entries 1  memlock 312B
        pids systemd-network(551)
15: ringbuf  name written_sysctls  flags 0x0
        key 0B  value 0B  max_entries 262144  memlock 275776B
        pids systemd-network(551)
17: array  name .rodata.str1.1  flags 0x80
        key 4B  value 5B  max_entries 1  memlock 312B
        frozen
        pids systemd-network(551)
34: array  name kprobe_b.bss  flags 0x400
        key 4B  value 4B  max_entries 1  memlock 8192B
        btf_id 258
        pids kprobe(1721748)
35: array  name kprobe_b.rodata  flags 0x80
        key 4B  value 133B  max_entries 1  memlock 440B
        btf_id 258  frozen
        pids kprobe(1721748)
38: array  name libbpf_global  flags 0x0
        key 4B  value 32B  max_entries 1  memlock 336B
        btf_id 258
39: array  name pid_iter.rodata  flags 0x480
        key 4B  value 4B  max_entries 1  memlock 8192B
        btf_id 269  frozen
        pids bpftool(1721865)
40: array  name libbpf_det_bind  flags 0x0
        key 4B  value 32B  max_entries 1  memlock 336B
        btf_id 269
```

Design#3: Variable Library

- eBPF Map 算什么?
- 资源视角?
- 是不是很像 pid?
- 只需要借助 eBPF Namespace 管理不同的 map/prog/link 视图

```
+mrzleo@turingine ~ % sudo bpftool map
12: hash_of_maps  name cgroup_hash  flags 0x0
        key 8B  value 4B  max_entries 2048  memlock 165632B
        pids systemd(1)
14: cgroup_array  name cgroup_map  flags 0x0
        key 4B  value 4B  max_entries 1  memlock 312B
        pids systemd-network(551)
15: ringbuf  name written_sysctls  flags 0x0
        key 0B  value 0B  max_entries 262144  memlock 275776B
        pids systemd-network(551)
17: array  name .rodata.str1.1  flags 0x80
        key 4B  value 5B  max_entries 1  memlock 312B
        frozen
        pids systemd-network(551)
34: array  name kprobe_b.bss  flags 0x400
        key 4B  value 4B  max_entries 1  memlock 8192B
        btf_id 258
        pids kprobe(1721748)
35: array  name kprobe_b.rodata  flags 0x80
        key 4B  value 133B  max_entries 1  memlock 440B
        btf_id 258  frozen
        pids kprobe(1721748)
38: array  name libbpf_global  flags 0x0
        key 4B  value 32B  max_entries 1  memlock 336B
        btf_id 258
39: array  name pid_iter.rodata  flags 0x480
        key 4B  value 4B  max_entries 1  memlock 8192B
        btf_id 269  frozen
        pids bpftool(1721865)
40: array  name libbpf_det_bind  flags 0x0
        key 4B  value 32B  max_entries 1  memlock 336B
        btf_id 269
```

Design#3: Variable Library

- 因此，eBPF- related state 的隔离是 vBPF builtin 保证的
- 我们现在只需要考虑 non-eBPF state
- 仍然从 tenant-private 和 true sharing 的角度分析

Design#3: Variable Library

- 把原本全局的变量转化成私有的
- 矛盾：变量是静态定义的，而 tenant 是动态变化的
- Tenant-private 的声明式 API
- 底层由 vBPF 自动化管理

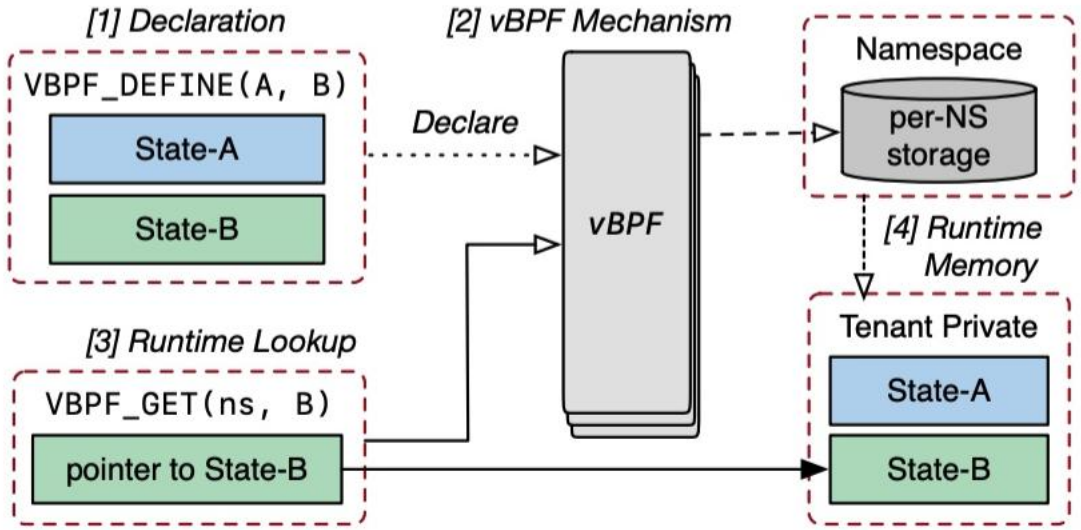


Figure 6: vBPF **tenant-private states.** *Developers declare multiple states using* VBPF_DEFINE *([1]), and then* vBPF *automatically maps them into a cohesive per-namespace storage layout ([2]). At runtime,* VBPF_GET *efficiently resolves ([3]) the specific state instance from the tenant-private memory ([4]).*

Design#3: Variable Library

- 真共享的情况下，怎么做隔离?
- 核心：不同 tenant 看到不同的视图
- 语义 Overlay：运行时隔离
- Tenant 拥有自己的 patchset
- 在切换的时候 restore/apply

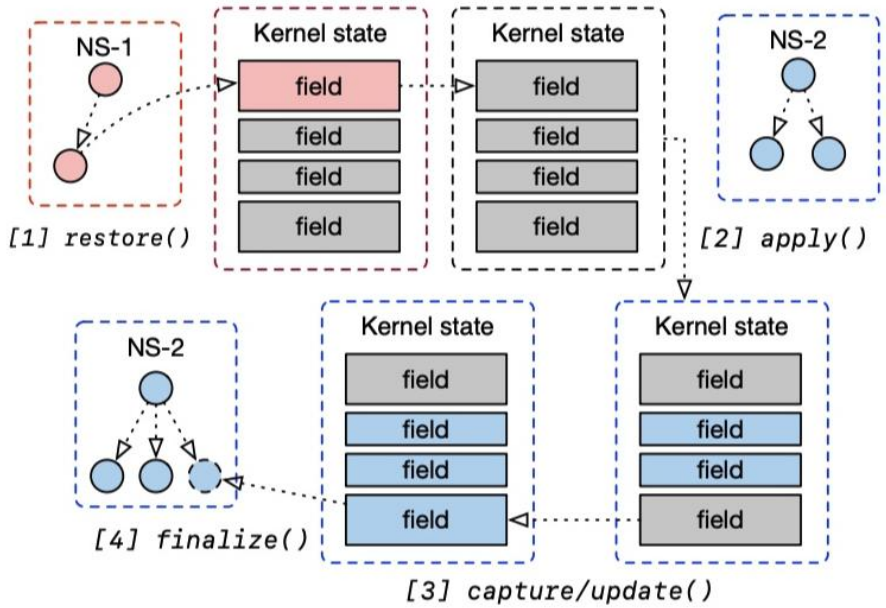


Figure 7: vBPF **semantic state overlay.** *Upon a context switch from NS-1 to NS-2,* vBPF *restores the kernel to a clean state ([1]) before applying NS-2's specific patches ([2]). During eBPF program execution, helper functions capture modifications ([3]), which are finalized and appended to the namespace's patch set ([4]) after the program exits.*

Design#3: Variable Library

- 怎么记录修改?
- 单纯的memory copy不够
- 需要按字段和语义打补丁
- 怎么在运行时知道字段和语义?

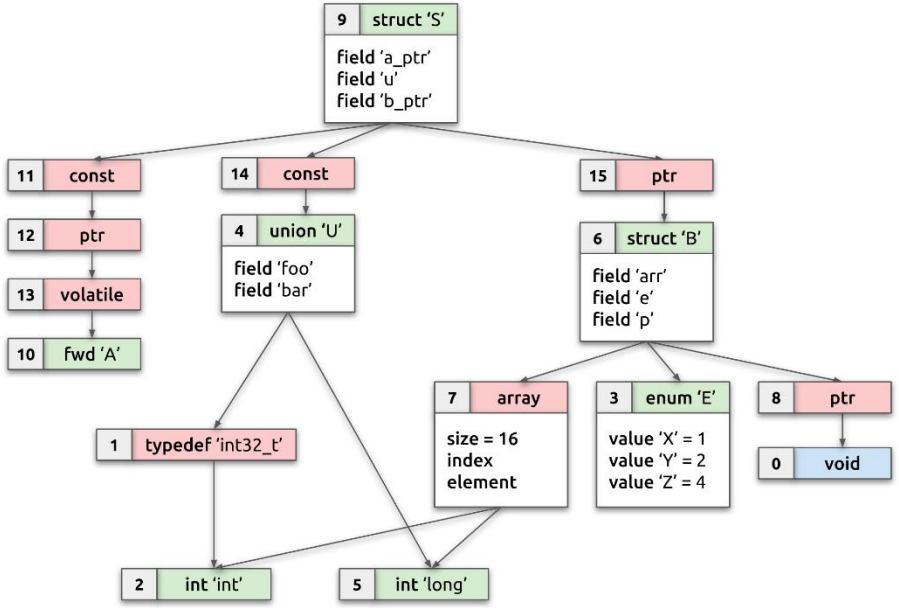
Design#3: Variable Library

- 怎么记录修改?
- 单纯的memory copy不够
- 需要按字段和语义打补丁
- 怎么在运行时知道字段和语义?
- 我们需要的其实是 反射

Design#3: Variable Library

- 怎么记录修改?
- 单纯的memory copy不够
- 需要按字段和语义打补丁
- 怎么在运行时知道字段和语义?
- 我们需要的其实是 反射

```
typedef int int32_t;

enum E {
    X = 1,
    Y = 2,
    Z = 4
};

union U {
    int32_t foo;
    long bar;
};

struct A;

struct B {
    long arr[16];
    enum E e;
    void* p;
};

struct S {
    volatile struct A* const a_ptr;
    const union U u;
    struct B* b_ptr;
};

int main() {
    struct S s;
    return 0;
}
```



https://facebookmicrosites.github.io/bpf/blog/2018/11/14/btf-enhancement.html

Design#3: Variable Library

- vBPF Overlay API
- Init 阶段提前解析并缓存
- 运行时仅进行内存对比和拷贝

```
// Precompute the context
struct overlay_ctx ctx;
overlay_ctx_init(&ctx, "sk_buff", skb);
// Hot path: record the patches
struct patch_builder *pb;
pb = overlay_begin(&ctx);
{ skb->priority++; }
overlay_capture(pb, skb);
/* Or use update */
overlay_update(pb, skb, struct sk_buff, tstamp);
overlay_finalize(&ctx, pb);
```

Listing 2: **Primitive usage in helper function.**

Take Away

- vBPF 采用了延迟绑定的模型，打破了传统静态绑定的缺陷
- Sniffer 通过预先建立的映射关系保证在中断上下文中的身份识别
- Dispatcher 在 hook 触发时动态选择执行
- State Isolation Framework 结合编译器和运行时保证状态隔离

谢 谢 大 家

陕西・西安 2026年4月18日